LAPORAN PRAKTIKUM

POSTTEST(3)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Nama:Isrando MIrhjrah (2509106093)

Kelas (C”25)

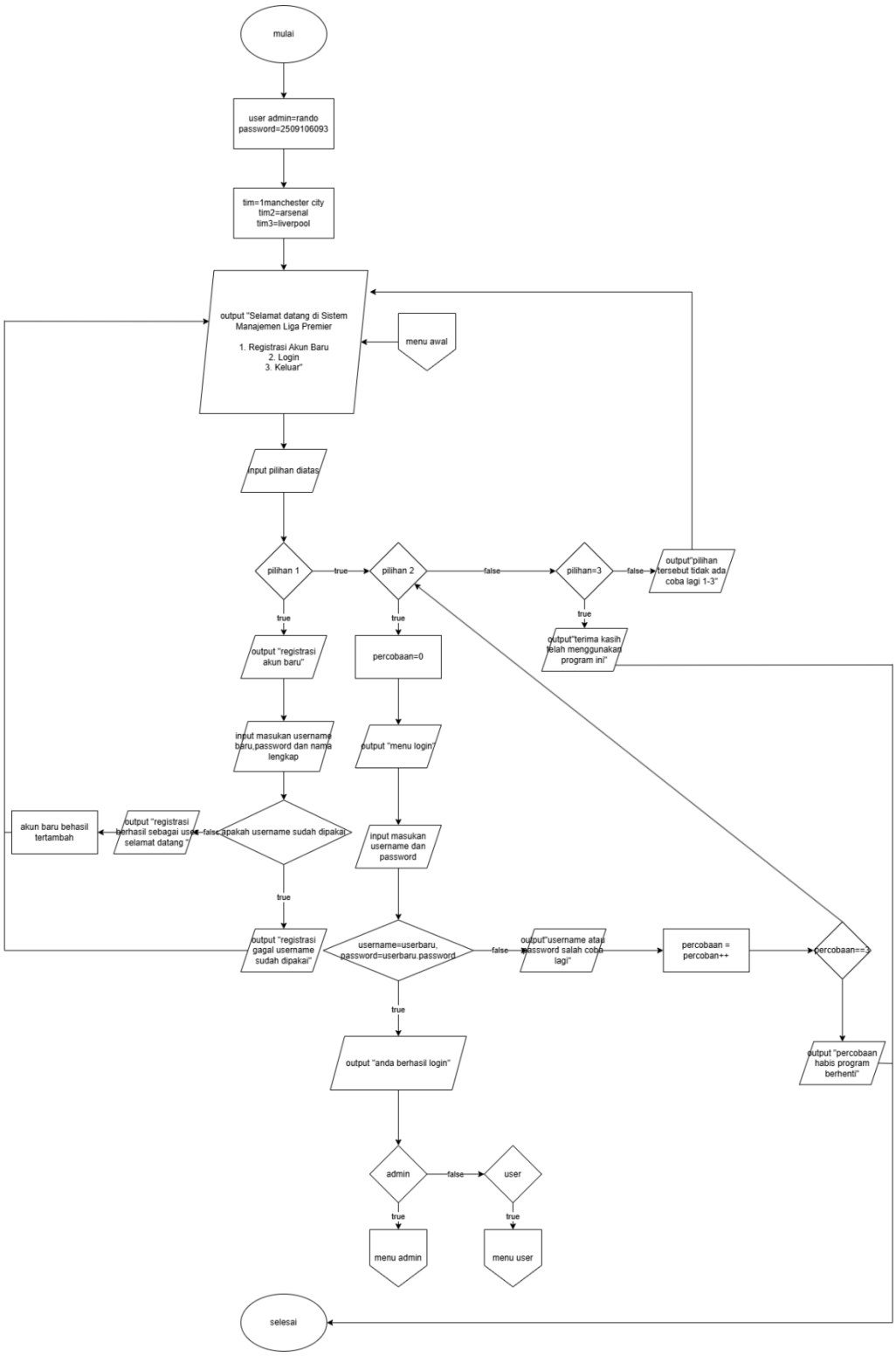
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

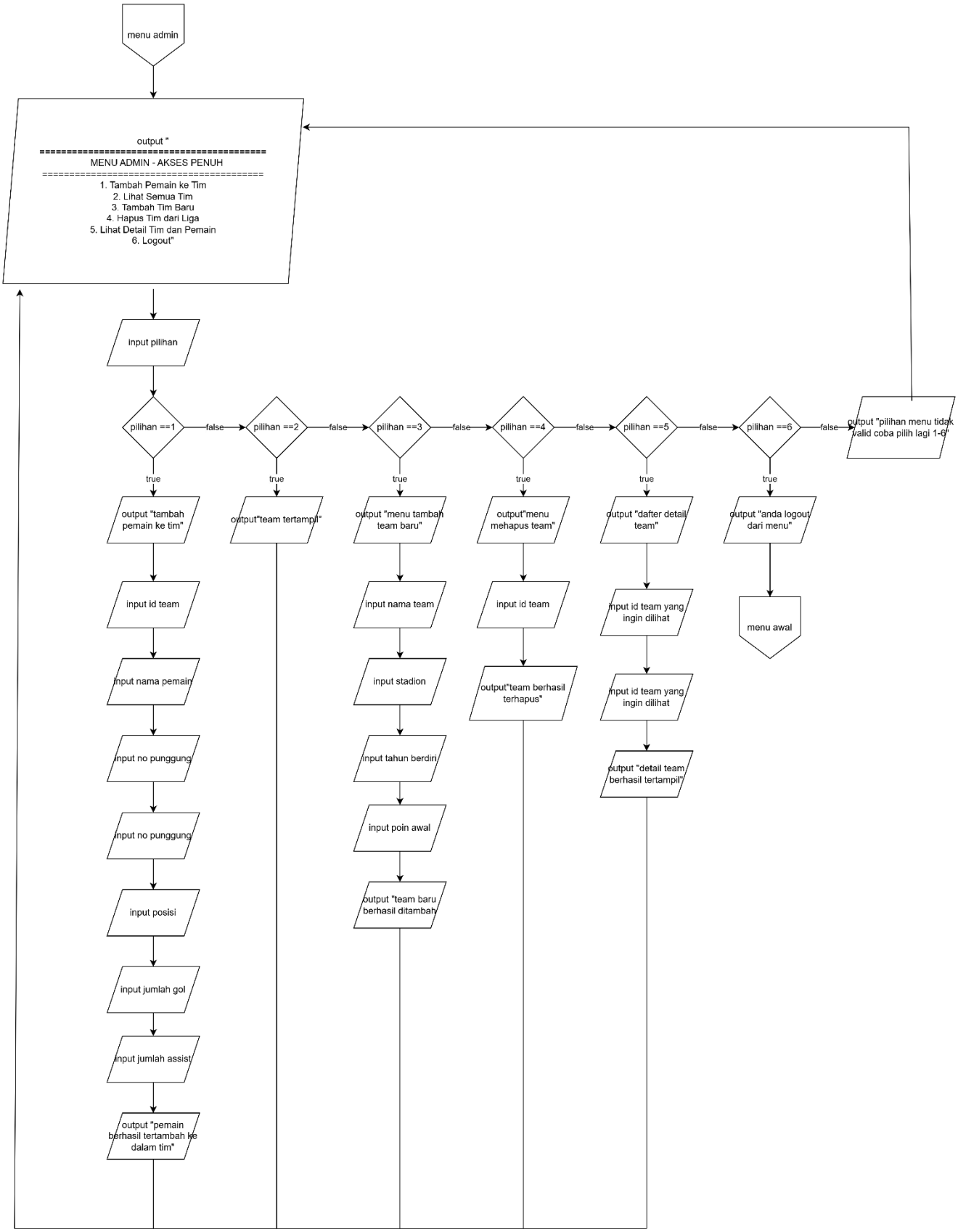
SAMARINDA

2025

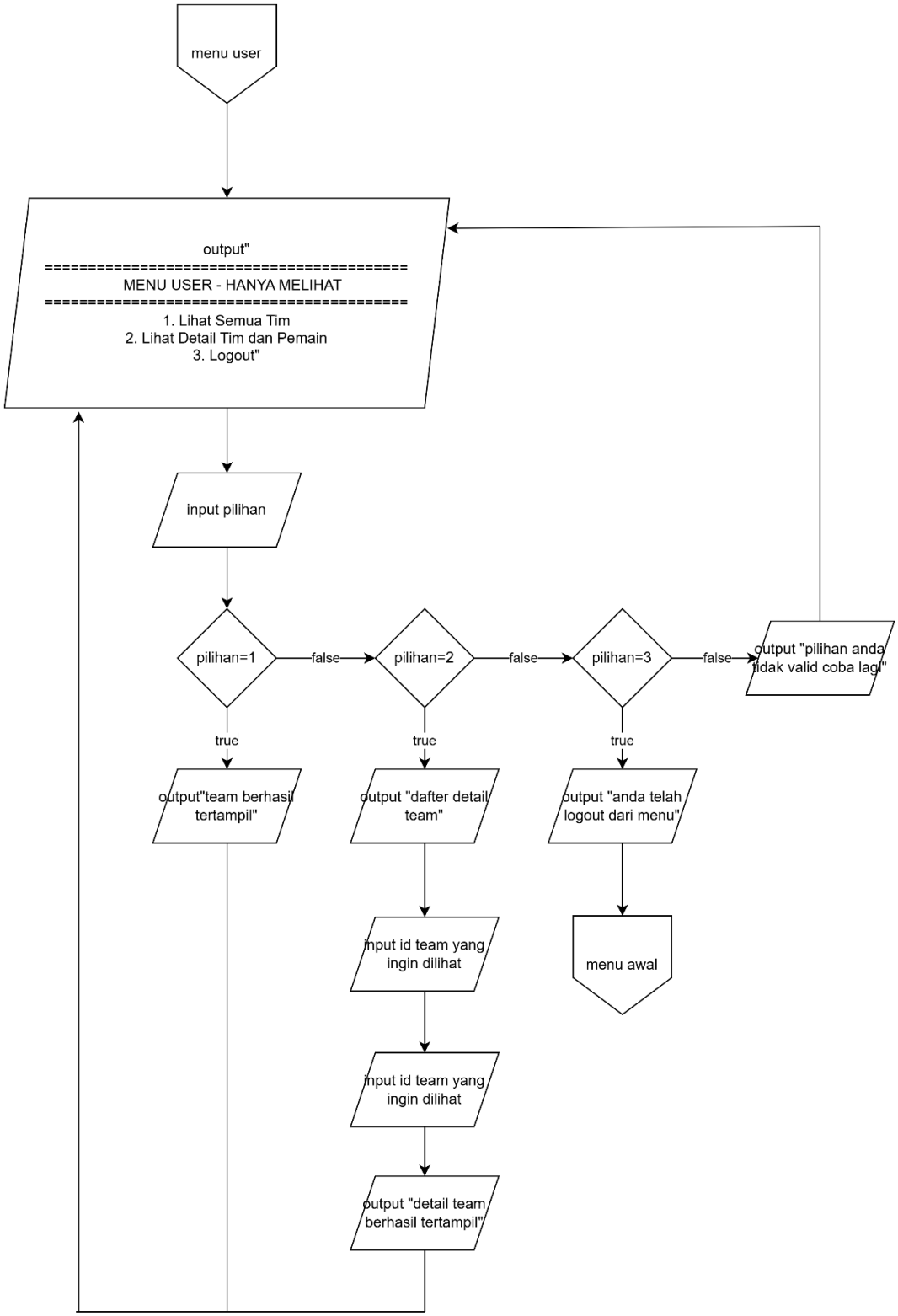
1. Flowchart



Flowchart 1.1 main menu



Flowchart 1.2 menu admin



Flowchart 1.3 menu user

Flowchart dia atas untuk menggambarkan cara alur program yang telah dibuat

2. Deskripsi Singkat Program

Progam ini dibuat untuk agar kita bisa melihat detail team yang ada di premier league dan kita bisa menambahkan team baru dan menghapus team lama,kita juga bisa menambahkan pemain ke dalam team

3. Source Code

```
#include <iostream>
#include <iomanip>
#include <string>
#include <map>
#include <limits>
#include <vector>
using namespace std;

struct Pemain {
    string namaPemain;
    int nomorPunggung;
    string posisi;
    int gol;
    int assist;
};

struct Tim {
    int id;
    string namaTim;
    string stadion;
    int tahunBerdiri;
    int poin;
    Pemain skuad[11];
    int jumlahPemain;
};

struct User {
    string username;
    string password;
    string role;
    string namaLengkap;
};

map<string, User> databaseUsers;
const int MAKS_TIM = 20;
Tim daftarTim[MAKS_TIM];
int jumlahTim = 0;
int nextIdTim = 1;

void clearScreen() {
    #ifdef _WIN32
        system("cls");
    #else
        system("clear");
    #endif
}

void tampilkanHeader(string judul) {
    cout << "\n==========================================\n";
    cout << "    " << judul << "\n";
    cout << "==========================================\n";
}
```

```
void tampilkanGaris(char ch = '=', int panjang = 50) {
    cout << string(panjang, ch) << endl;
}

void tampilkanPesan(string pesan, bool tungguEnter = true) {
    cout << pesan << endl;
    if (tungguEnter) {
        cout << "Tekan Enter untuk melanjutkan...";
        cin.get();
    }
}

int cariTim(int idTim, int jumlahTim, Tim daftarTim[]) {
    for (int i = 0; i < jumlahTim; i++) {
        if (daftarTim[i].id == idTim) {
            return i;
        }
    }
    return -1;
}

int cariTim(string namaTim, int jumlahTim, Tim daftarTim[]) {
    for (int i = 0; i < jumlahTim; i++) {
        if (daftarTim[i].namaTim == namaTim) {
            return i;
        }
    }
    return -1;
}

void infoTim(Tim tim) {
    cout << "ID: " << tim.id << " | " << tim.namaTim
         << " (Pemain: " << tim.jumlahPemain << "/11, Poin: " <<
tim.poin << ")";
}

void infoTim(Tim tim, bool detailLengkap) {
    if (detailLengkap) {
        cout << "\n=== DETAIL TIM ===\n";
        cout << "ID Tim          : " << tim.id << "\n";
        cout << "Nama Tim        : " << tim.namaTim << "\n";
        cout << "Stadion         : " << tim.stadion << "\n";
        cout << "Tahun Berdiri   : " << tim.tahunBerdiri << "\n";
        cout << "Poin            : " << tim.poin << "\n";
        cout << "Jumlah Pemain   : " << tim.jumlahPemain << "/11\n";
    } else {
        infoTim(tim);
    }
}


int hitungTotalGol(Tim tim, int indeksPemain = 0) {
```

```
    if (indeksPemain >= tim.jumlahPemain) {
        return 0;
    }
    return tim.skuad[indeksPemain].gol + hitungTotalGol(tim,
indeksPemain + 1);
}


int hitungTotalAssist(Tim tim, int indeksPemain = 0) {
    if (indeksPemain >= tim.jumlahPemain) {
        return 0;
    }
    return tim.skuad[indeksPemain].assist + hitungTotalAssist(tim,
indeksPemain + 1);
}


Pemain cariTopScorer(Tim tim, int indeksPemain = 0, Pemain topScorer =
{}) {
    if (indeksPemain >= tim.jumlahPemain) {
        return topScorer;
    }

    if (indeksPemain == 0 || tim.skuad[indeksPemain].gol >
topScorer.gol) {
        topScorer = tim.skuad[indeksPemain];
    }

    return cariTopScorer(tim, indeksPemain + 1, topScorer);
}


void tampilkanPemain(Tim tim, int indeksPemain = 0) {
    if (indeksPemain >= tim.jumlahPemain) {
        return;
    }

    cout << left
         << setw(5) << indeksPemain + 1
         << setw(25) << tim.skuad[indeksPemain].namaPemain.substr(0, 24)
         << setw(15) << tim.skuad[indeksPemain].nomorPunggung
         << setw(15) << tim.skuad[indeksPemain].posisi.substr(0, 14)
         << setw(10) << tim.skuad[indeksPemain].gol
         << setw(10) << tim.skuad[indeksPemain].assist
         << "\n";

    tampilkanPemain(tim, indeksPemain + 1);
}


bool validasiTahunBerdiri(int tahun) {
    return (tahun >= 1800 && tahun <= 2024);
}
```

```
bool validasiNomorPunggung(int nomor, Tim tim) {
    if (nomor < 1 || nomor > 99) return false;

    for (int i = 0; i < tim.jumlahPemain; i++) {
        if (tim.skuad[i].nomorPunggung == nomor) {
            return false;
        }
    }
    return true;
}


int inputAngka(string pesan, int minValue, int maxValue) {
    int angka;
    while (true) {
        cout << pesan;
        cin >> angka;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Input harus berupa angka! Silakan coba lagi.\n";
        } else if (angka < minValue || angka > maxValue) {
            cout << "Input harus antara " << minValue << " - " <<
maxValue << "! Silakan coba lagi.\n";
        } else {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return angka;
        }
    }
}


int inputAngka(string pesan) {
    return inputAngka(pesan, INT_MIN, INT_MAX);
}

bool registrasi(map<string, User> &database, User &userBaru) {
    if (database.find(userBaru.username) != database.end()) {
        return false;
    }

    if (userBaru.password.length() < 6) {
        return false;
    }

    userBaru.role = "User";

    database[userBaru.username] = userBaru;
    return true;
}
```

```
bool loginUser(map<string, User> &database, string username, string
password, User &currentUser) {
    auto it = database.find(username);
    if (it != database.end() && it->second.password == password) {
        currentUser = it->second;
        return true;
    }
    return false;
}

bool tambahTimBaru(Tim daftarTim[], int &jumlahTim, int &nextId, Tim
timBaru) {
    if (jumlahTim >= MAKS_TIM) {
        return false;
    }

    timBaru.id = nextId++;
    timBaru.jumlahPemain = 0;
    daftarTim[jumlahTim] = timBaru;
    jumlahTim++;
    return true;
}


bool tambahPemainBaru(Tim &tim, Pemain pemainBaru) {
    if (tim.jumlahPemain >= 11) {
        return false;
    }

    tim.skuad[tim.jumlahPemain] = pemainBaru;
    tim.jumlahPemain++;
    return true;
}


bool hapusTimDariLiga(Tim daftarTim[], int &jumlahTim, int idTim) {
    int indeks = cariTim(idTim, jumlahTim, daftarTim);
    if (indeks == -1) {
        return false;
    }

    for (int i = indeks; i < jumlahTim - 1; i++) {
        daftarTim[i] = daftarTim[i + 1];
    }
    jumlahTim--;
    return true;
}


void tampilkanStatistikTim(Tim tim) {
    cout << "\n=== STATISTIK TIM: " << tim.namaTim << " ===\n";
    cout << "Total Gol Semua Pemain  : " << hitungTotalGol(tim) << endl;
    cout << "Total Assist Semua Pemain : " << hitungTotalAssist(tim) <<
```

```
endl;

    if (tim.jumlahPemain > 0) {
        Pemain topScorer = cariTopScorer(tim);
        cout << "Top Scorer              : " << topScorer.namaPemain
             << " (" << topScorer.gol << " gol)" << endl;
    }
}



void registrasi() {
    clearScreen();
    tampilkanHeader("REGISTRASI AKUN BARU");

    User userBaru;

    cout << "Masukkan Username: ";
    getline(cin, userBaru.username);
    cout << "Masukkan Password (minimal 6 karakter): ";
    getline(cin, userBaru.password);
    cout << "Masukkan Nama Lengkap: ";
    getline(cin, userBaru.namaLengkap);


    if (registrasi(databaseUsers, userBaru)) {
        cout << "\n==========================================\n";
        cout << "         REGISTRASI BERHASIL!\n";
        cout << "==========================================\n";
        cout << "Username    : " << userBaru.username << "\n";
        cout << "Nama        : " << userBaru.namaLengkap << "\n";
        cout << "Role        : User \n";
        cout << "==========================================\n";
        cout << "Silakan login menggunakan akun Anda.\n";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
    } else {
        cout << "\n==========================================\n";
        cout << "           REGISTRASI GAGAL!\n";
        cout << "==========================================\n";
        if (databaseUsers.find(userBaru.username) !=
databaseUsers.end()) {
            cout << "Username '" << userBaru.username << "' sudah
digunakan!\n";
        } else if (userBaru.password.length() < 6) {
            cout << "Password harus minimal 6 karakter!\n";
        } else {
            cout << "Terjadi kesalahan saat registrasi.\n";
        }
        cout << "==========================================\n";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
    }
```

```
}

bool login(User &currentUser) {
    string username, password;
    int percobaan = 0;
    const int maxPercobaan = 3;

    while (percobaan < maxPercobaan) {
        clearScreen();
        tampilkanHeader("MENU LOGIN");
        cout << "Username: ";
        getline(cin, username);
        cout << "Password: ";
        getline(cin, password);


        auto it = databaseUsers.find(username);
        if (it != databaseUsers.end() && it->second.password ==
password) {
            currentUser = it->second;

            tampilkanGaris('-');
            cout << "\nLOGIN BERHASIL!\n";
            cout << "Selamat datang, " << currentUser.namaLengkap <<
"!\n";
            cout << "Role: " << currentUser.role << "\n";
            tampilkanGaris('-');

            cout << "\nTekan Enter untuk melanjutkan...";
            cin.get();
            return true;
        }

        percobaan++;
        if (percobaan < maxPercobaan) {
            cout << "\nUsername atau password salah! Silakan coba
lagi.\n";
            cout << "Tekan Enter untuk melanjutkan...";
            cin.get();
        }

    }
    clearScreen();
    tampilkanHeader("LOGIN GAGAL");
    cout << "\nAnda telah gagal login 3 kali.\n";
    cout << "Tekan Enter untuk keluar...";
    cin.get();
    exit (0);
    return false;
}

void lihatSemuaTim() {
    clearScreen();
```

```
    tampilkanHeader("DAFTAR SEMUA TIM");

    if (jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    cout << left
         << setw(5) << "ID"
         << setw(25) << "Nama Tim"
         << setw(25) << "Stadion"
         << setw(15) << "Tahun"
         << setw(10) << "Poin"
         << setw(15) << "Pemain"
         << "\n";
    tampilkanGaris('-', 95);

    for (int i = 0; i < jumlahTim; i++) {
        cout << left
             << setw(5) << daftarTim[i].id
             << setw(25) << daftarTim[i].namaTim.substr(0, 24)
             << setw(25) << daftarTim[i].stadion.substr(0, 24)
             << setw(15) << daftarTim[i].tahunBerdiri
             << setw(10) << daftarTim[i].poin
             << setw(15) << daftarTim[i].jumlahPemain
             << "\n";
    }
    tampilkanGaris('=', 95);

    cout << "\nTekan Enter untuk kembali...";
    cin.get();
}

void lihatDetailTim() {
    clearScreen();
    tampilkanHeader("DETAIL TIM DAN PEMAIN");

    if (jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    lihatSemuaTim();

    int idTim = inputAngka("\nMasukkan ID Tim yang ingin dilihat
detailnya: ", 1, nextIdTim - 1);

    int indeksTim = cariTim(idTim, jumlahTim, daftarTim);

    if (indeksTim == -1) {
        tampilkanPesan("Tim dengan ID " + to_string(idTim) + " tidak
ditemukan!", true);
        return;
```

```
    }

    clearScreen();
    infoTim(daftarTim[indeksTim], true);

    if (daftarTim[indeksTim].jumlahPemain == 0) {
        cout << "\nBelum ada pemain di tim ini.\n";
    } else {
        cout << "\nDAFTAR PEMAIN:\n";
        tampilkanGaris('-', 70);
        cout << left
             << setw(5) << "No"
             << setw(25) << "Nama Pemain"
             << setw(15) << "No Punggung"
             << setw(15) << "Posisi"
             << setw(10) << "Gol"
             << setw(10) << "Assist"
             << "\n";
        tampilkanGaris('-', 70);


        tampilkanPemain(daftarTim[indeksTim]);
        tampilkanGaris('=', 70);


        tampilkanStatistikTim(daftarTim[indeksTim]);
    }

    cout << "\nTekan Enter untuk kembali...";
    cin.get();
}

void tambahTim() {
    clearScreen();
    tampilkanHeader("TAMBAH TIM BARU");

    if (jumlahTim >= MAKS_TIM) {
        tampilkanPesan("Database tim sudah penuh (maksimal " +
to_string(MAKS_TIM) + " tim)!", true);
        return;
    }

    Tim timBaru;

    cout << "Nama Tim: ";
    getline(cin, timBaru.namaTim);
    cout << "Stadion: ";
    getline(cin, timBaru.stadion);

    do {
        timBaru.tahunBerdiri = inputAngka("Tahun Berdiri: ");
        if (!validasiTahunBerdiri(timBaru.tahunBerdiri)) {
            cout << "Tahun harus antara 1800 - 2024!\n";
```

```
        }
    } while (!validasiTahunBerdiri(timBaru.tahunBerdiri));

    timBaru.poin = inputAngka("Poin Awal: ", 0, 100);

    if (tambahTimBaru(daftarTim, jumlahTim, nextIdTim, timBaru)) {
        tampilkanPesan("\nTim " + timBaru.namaTim + " berhasil
ditambahkan! ID Tim: " +
                       to_string(timBaru.id), true);
    } else {
        tampilkanPesan("\nGagal menambahkan tim!", true);
    }
}

void tambahPemain() {
    clearScreen();
    tampilkanHeader("TAMBAH PEMAIN KE TIM");

    if (jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar. Tambah tim terlebih
dahulu.", true);
        return;
    }

    cout << "DAFTAR TIM:\n";
    tampilkanGaris('-', 50);
    for (int i = 0; i < jumlahTim; i++) {
        infoTim(daftarTim[i]);
        cout << endl;
    }
    tampilkanGaris('-', 50);

    int idTim = inputAngka("\nMasukkan ID Tim: ", 1, nextIdTim - 1);
    int indeksTim = cariTim(idTim, jumlahTim, daftarTim);

    if (indeksTim == -1) {
        tampilkanPesan("Tim dengan ID " + to_string(idTim) + " tidak
ditemukan!", true);
        return;
    }

    if (daftarTim[indeksTim].jumlahPemain >= 11) {
        tampilkanPesan("Tim sudah mencapai maksimal pemain (11
pemain)!", true);
        return;
    }

    cout << "\nMenambah pemain untuk tim: " <<
daftarTim[indeksTim].namaTim << "\n";
    cout << "Jumlah pemain saat ini: " <<
daftarTim[indeksTim].jumlahPemain << "/11\n\n";

    Pemain pemainBaru;
```

```
    cout << "Nama Pemain: ";
    getline(cin, pemainBaru.namaPemain);

    do {
        pemainBaru.nomorPunggung = inputAngka("Nomor Punggung (1-99): ",
1, 99);
        if (!validasiNomorPunggung(pemainBaru.nomorPunggung,
daftarTim[indeksTim])) {
            cout << "Nomor punggung sudah digunakan atau tidak
valid!\n";
        }
    } while (!validasiNomorPunggung(pemainBaru.nomorPunggung,
daftarTim[indeksTim]));

    cout << "Posisi: ";
    getline(cin, pemainBaru.posisi);
    pemainBaru.gol = inputAngka("Jumlah Gol: ", 0, 100);
    pemainBaru.assist = inputAngka("Jumlah Assist: ", 0, 100);

    if (tambahPemainBaru(daftarTim[indeksTim], pemainBaru)) {
        tampilkanPesan("\nPemain " + pemainBaru.namaPemain + " berhasil
ditambahkan ke " +
                       daftarTim[indeksTim].namaTim + "!", true);
    } else {
        tampilkanPesan("\nGagal menambahkan pemain!", true);
    }
}

void hapusTim() {
    clearScreen();
    tampilkanHeader("HAPUS TIM DARI LIGA");

    if (jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    lihatSemuaTim();

    int idTim = inputAngka("\nMasukkan ID Tim yang akan dihapus: ", 1,
nextIdTim - 1);

    int indeksHapus = cariTim(idTim, jumlahTim, daftarTim);
    if (indeksHapus == -1) {
        tampilkanPesan("\nTim dengan ID " + to_string(idTim) + " tidak
ditemukan!", true);
        return;
    }

    cout << "\nAnda akan menghapus tim:\n";
    cout << "Nama Tim    : " << daftarTim[indeksHapus].namaTim << "\n";
    cout << "Stadion     : " << daftarTim[indeksHapus].stadion << "\n";
    cout << "Jumlah Pemain: " << daftarTim[indeksHapus].jumlahPemain <<
```

```
" pemain\n\n";
    cout << "Yakin ingin menghapus tim ini? (y/n): ";

    char konfirmasi;
    cin >> konfirmasi;
    cin.ignore();

    if (konfirmasi == 'y' || konfirmasi == 'Y') {
        if (hapusTimDariLiga(daftarTim, jumlahTim, idTim)) {
            tampilkanPesan("\nTim berhasil dihapus dari liga!", true);
        } else {
            tampilkanPesan("\nGagal menghapus tim!", true);
        }
    } else {
        tampilkanPesan("\nPenghapusan dibatalkan.", true);
    }
}

void menuAdmin(User &currentUser) {
    int pilihan;
    bool running = true;

    do {
        clearScreen();
        tampilkanHeader("MENU ADMIN");
        cout << "User: " << currentUser.username << " | " <<
currentUser.namaLengkap << "\n\n";
        cout << "1. Tambah Pemain ke Tim\n";
        cout << "2. Lihat Semua Tim\n";
        cout << "3. Tambah Tim Baru\n";
        cout << "4. Hapus Tim dari Liga\n";
        cout << "5. Lihat Detail Tim dan Pemain\n";
        cout << "6. Logout\n\n";

        pilihan = inputAngka("Pilih menu (1-6): ", 1, 6);

        switch (pilihan) {
            case 1: tambahPemain(); break;
            case 2: lihatSemuaTim(); break;
            case 3: tambahTim(); break;
            case 4: hapusTim(); break;
            case 5: lihatDetailTim(); break;
            case 6:
                cout << "\nLogout berhasil!\n";
                running = false;
                break;
        }
    } while (running);
}

void menuUser(User &currentUser) {
    int pilihan;
    bool running = true;
```

```
    do {
        clearScreen();
        tampilkanHeader("MENU USER");
        cout << "User: " << currentUser.username << " | " <<
currentUser.namaLengkap << "\n\n";
        cout << "1. Lihat Semua Tim\n";
        cout << "2. Lihat Detail Tim dan Pemain\n";
        cout << "3. Logout\n\n";

        pilihan = inputAngka("Pilih menu (1-3): ", 1, 3);

        switch (pilihan) {
            case 1: lihatSemuaTim(); break;
            case 2: lihatDetailTim(); break;
            case 3:
                cout << "\nLogout berhasil!\n";
                running = false;
                break;
        }
    } while (running);
}

void inisialisasiData() {

    User admin;
    admin.username = "rando";
    admin.password = "2509106093";
    admin.namaLengkap = "Administrator";
    admin.role = "Admin";
    databaseUsers["rando"] = admin;


    User user;
    user.username = "rando1";
    user.password = "093";
    user.namaLengkap = "Biasa User";
    user.role = "User";
    databaseUsers["rando1"] = user;


    if (jumlahTim == 0) {
        Tim tim1;
        tim1.id = nextIdTim++;
        tim1.namaTim = "Manchester City";
        tim1.stadion = "Etihad Stadium";
        tim1.tahunBerdiri = 1880;
        tim1.poin = 73;
        tim1.jumlahPemain = 0;
        daftarTim[jumlahTim++] = tim1;

        Tim tim2;
        tim2.id = nextIdTim++;
```

```
        tim2.namaTim = "Arsenal";
        tim2.stadion = "Emirates Stadium";
        tim2.tahunBerdiri = 1886;
        tim2.poin = 71;
        tim2.jumlahPemain = 0;
        daftarTim[jumlahTim++] = tim2;

        Tim tim3;
        tim3.id = nextIdTim++;
        tim3.namaTim = "Liverpool";
        tim3.stadion = "Anfield";
        tim3.tahunBerdiri = 1892;
        tim3.poin = 70;
        tim3.jumlahPemain = 0;
        daftarTim[jumlahTim++] = tim3;
    }
}

void menuAwal() {
    int pilihan;
    bool programRunning = true;
    User currentUser;

    inisialisasiData();

    do {
        clearScreen();
        tampilkanHeader("SISTEM MANAJEMEN LIGA PREMIER");
        cout << "Selamat datang di Sistem Manajemen Liga Premier\n\n";
        cout << "1. Registrasi Akun Baru\n";
        cout << "2. Login\n";
        cout << "3. Keluar\n\n";

        pilihan = inputAngka("Pilih menu (1-3): ", 1, 3);

        switch (pilihan) {
            case 1:
                registrasi();
                break;
            case 2:
                if (login(currentUser)) {
                    if (currentUser.role == "Admin") {
                        menuAdmin(currentUser);
                    } else {
                        menuUser(currentUser);
                    }
                }
                break;
            case 3:
                cout << "\nTerima kasih telah menggunakan program
ini!\n";
                programRunning = false;
                break;
```

```
        }
    } while (programRunning);
}

int main() {
    menuAwal();
    return 0;
}
```

4. Hasil Output

```
==========================================
   SISTEM MANAJEMEN LIGA PREMIER
==========================================
Selamat datang di Sistem Manajemen Liga Premier

1. Registrasi Akun Baru
2. Login
3. Keluar

Pilih menu (1-3):
```

Gambar 4.1 output menu awal

```
==========================================
   REGISTRASI AKUN BARU
==========================================
Masukkan Username: randooioi
Masukkan Password (minimal 6 karakter): randoioio
Masukkan Nama Lengkap: rando

==========================================
         REGISTRASI BERHASIL!
==========================================
Username     : randooioi
Nama         : rando
Role         : User
==========================================
Silakan login menggunakan akun Anda.

Tekan Enter untuk melanjutkan...
```

Gambar 4.2 output registasi akun sebagai user

```
==========================================
   MENU LOGIN
==========================================
Username: rando123
Password: rando123
--------------------------------------------------

LOGIN BERHASIL!
Selamat datang, rando!
Role: Admin
--------------------------------------------------

Tekan Enter untuk melanjutkan...
```

Gambar 4.3 output login menu berhasil

```
==========================================
   LOGIN GAGAL
==========================================

Anda telah gagal login 3 kali.
Tekan Enter untuk keluar...
PS D:\praktikum-apl\post-test\post-test-2>
```

Gambar 4.4 output ini ketika kita salah password hingga 3 kali

```
==========================================
   MENU ADMIN - AKSES PENUH
==========================================
User: rando | Administrator

1. Tambah Pemain ke Tim
2. Lihat Semua Tim
3. Tambah Tim Baru
4. Hapus Tim dari Liga
5. Lihat Detail Tim dan Pemain
6. Logout

Pilih menu (1-6): 
```

Gambar 4.5 output menu admin

```
==========================================
   TAMBAH PEMAIN KE TIM
==========================================
DAFTAR TIM:
--------------------------------------------------
ID: 1 | Manchester City (Pemain: 0/11)
ID: 2 | Arsenal (Pemain: 0/11)
ID: 3 | Liverpool (Pemain: 0/11)
--------------------------------------------------

Masukkan ID Tim: 2

Menambah pemain untuk tim: Arsenal
Jumlah pemain saat ini: 0/11

Nama Pemain: rando
Nomor Punggung: 10
Posisi: st
Jumlah Gol: 9
Jumlah Assist: 10

Pemain rando berhasil ditambahkan ke Arsenal!
Tekan Enter untuk kembali...
```

Gambar 4.6 output untuk menambahkan pemain ke team

```
==========================================
   DAFTAR SEMUA TIM
==========================================
ID   Nama Tim                Stadion                 Tahun Berdiri  Poin      Jumlah Pemain
------------------------------------------------------------------------------------------
1    Manchester City         Etihad Stadium          1880           73        0
2    Arsenal                 Emirates Stadium        1886           71        1
3    Liverpool               Anfield                 1892           70        0
==========================================================================================

Tekan Enter untuk kembali...
```

Gambar 4.7 output untuk melihat semua team dan poin

```
==========================================
   TAMBAH TIM BARU
==========================================
Nama Tim: barcelona
Stadion: segiri
Tahun Berdiri: 1945
Poin Awal: 89

Tim barcelona berhasil ditambahkan!
ID Tim: 4
Tekan Enter untuk kembali...
```

Gambar 4.8 output untuk menambahkan team baru ke liga

```
==========================================
   HAPUS TIM DARI LIGA
==========================================
DAFTAR TIM:
----------------------------------------------------------------------
ID   Nama Tim                Stadion                 Poin      Jumlah Pemain
----------------------------------------------------------------------
1    Manchester City         Etihad Stadium          73        0
2    Arsenal                 Emirates Stadium        71        1
3    Liverpool               Anfield                 70        0
4    barcelona               segiri                  89        0
======================================================================

Masukkan ID Tim yang akan dihapus: 1

Anda akan menghapus tim:
Nama Tim    : Manchester City
Stadion     : Etihad Stadium
Jumlah Pemain: 0 pemain

Yakin ingin menghapus tim ini? (y/n): y

Tim Manchester City berhasil dihapus dari liga!

Tekan Enter untuk kembali...
```

Gambar 4.9 output untuk menghapus team

```
==========================================
   DETAIL TIM
==========================================
ID Tim         : 2
Nama Tim       : Arsenal
Stadion        : Emirates Stadium
Tahun Berdiri  : 1886
Poin           : 71
Jumlah Pemain  : 1/11

DAFTAR PEMAIN:
----------------------------------------------------------------------
No   Nama Pemain             No Punggung    Posisi        Gol       Assist
----------------------------------------------------------------------
1    rando                   10             st            9         10
======================================================================

Tekan Enter untuk kembali...
```

Gambar 5.0 output ingin melihat detail team

```
==========================================
   MENU USER - HANYA MELIHAT
==========================================
User: rando1 | Biasa User

1. Lihat Semua Tim
2. Lihat Detail Tim dan Pemain
3. Logout

Pilih menu (1-3):
```

Gambar 5.1 output menu user

```
========================================
   DAFTAR SEMUA TIM
========================================
ID   Nama Tim                Stadion                  Tahun Berdiri  Poin      Jumlah Pemain
----------------------------------------------------------------------------------------------
2    Arsenal                 Emirates Stadium         1886           71        1
3    Liverpool               Anfield                  1892           70        0
4    barcelona               segiri                   1945           89        0
==============================================================================================

Tekan Enter untuk kembali...
```

Gambar 5.2 output untuk melihat semua team dan poin

```
==========================================
   DETAIL TIM
==========================================
ID Tim          : 2
Nama Tim        : Arsenal
Stadion         : Emirates Stadium
Tahun Berdiri   : 1886
Poin            : 71
Jumlah Pemain   : 1/11

DAFTAR PEMAIN:
----------------------------------------------------------------------
No   Nama Pemain              No Punggung    Posisi         Gol       Assist
----------------------------------------------------------------------
1    rando                    10             st             9         10
======================================================================

Tekan Enter untuk kembali...
```

Gambar 5.3 output ingin melihat detail team

```
==========================================
   SISTEM MANAJEMEN LIGA PREMIER
==========================================
Selamat datang di Sistem Manajemen Liga Premier

1. Registrasi Akun Baru
2. Login
3. Keluar

Pilih menu (1-3): 3

Terima kasih telah menggunakan program ini!
PS D:\praktikum-apl\post-test\post-test-2>
```

Gambar 5.4 output untuk keluar dari program

5. Langkah-langkah GIT

5.1 GIT Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.1 git add itu dipakai untuk menandai file yang berubah supaya siap disimpan ke Git dengan git commit.

5.2 GIT Commit

```
PS D:\praktikum-apl> git commit -m "posttest3"
```

Gambar 5.2 git commit itu kayak **menyimpan catatan resmi** dari perubahan yang sudah kamu siapkan dengan git add.

5.3 GIT Push

```
PS D:\praktikum-apl> git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 698.93 KiB | 8.13 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/israndomirhjrah/praktikum-apl.git
   c7b7b6f..ad5565f  main -> main
```

Gambar 5.3 git push itu dipakai untuk **mengirim commit yang ada di komputer lokal ke server Git (misalnya GitHub, GitLab, atau Bitbucket)**. Jadi perubahan yang sudah kamu simpan dengan git commit akan dikirim ke repository online supaya bisa dilihat atau dipakai orang lain.